

CA3: Reflective Document

Crush Arena: Networked Vertical Slice with NPC Inclusion

Advanced 3D Game Development

Section 1: Rendering Foundations

The rendering foundations established in CA1 were carried forward into the CA3 vertical slice. The Crush Arena scene uses the same PBR discipline and dusk lighting approach developed for the Jade Toad demonstration, adapted to the arena environment.

PBR Material Discipline

The same colour-space rules from CA1 apply throughout the project as it was used as a baseline. Colour maps (Albedo, Emission) are imported with sRGB enabled for correct gamma-to-linear conversion during rendering. Data maps (Normal, MetallicSmoothness, AO, Height) are imported with sRGB disabled since they store linear numerical values that would be distorted by an incorrect gamma curve. Normal maps use the Normal Map texture type for correct compression and tangent-space handling.

For the arena, materials are kept simple, solid colours for walls, floor, platforms, and crushers — since the focus of CA3 is networking and gameplay.

Lighting and Post-Processing

The arena uses a dusk lighting setup consistent with the CA1 approach. A low-angle warm directional light (Key_DuskSun) provides the primary illumination.

A Global Volume provides post-processing with ACES tonemapping, Bloom, and Vignette.

Unreal Engine Comparison Note (Updated)

The core observations from CA1 remain valid. Unity uses separate texture maps (Albedo, AO, MetallicSmoothness) while Unreal packs AO, Roughness, and Metallic into a single ORM texture. Unity uses Smoothness (1 = smooth) while Unreal uses Roughness (1 = rough); these are inverses. Normal maps differ in convention: Unity defaults to OpenGL (Y+ up), Unreal uses DirectX (Y+ down).

My understanding has only minorly shifted in any through the networking work in CA2 and CA3 as their was minimal to no interaction with Unreal relative to CA1.

Section 2: Optimisation and Constraints

Profiling Evidence

///

Constraint Decisions

The vertical slice uses simple primitive geometry (cubes, planes) for the arena, which keeps draw calls low and avoids the need for LOD groups or occlusion culling. The crushers are single-mesh cubes with NavMeshAgent components. The main rendering cost comes from dynamic shadows and the post-processing stack, both of which are controlled through the Global Volume and light settings.

For a production version, I would add LOD groups to any complex meshes, implement occlusion culling for the arena walls, and consider baking static shadows for the arena geometry while keeping only the dynamic objects (crushers, players) on realtime shadows.

Section 3: Networking Implementation

Feature Description

Crush Arena is a two-player networked game built with Photon Fusion 2 in Shared Mode. Two players spawn on elevated platforms on opposite sides of an arena. Collectible objects are scattered across the arena floor. Players drop into the arena to collect them, scoring points for each pickup. NPC "crushers, large blocks that pathfind toward players using NavMesh and slam down like Thwomps from Super Mario, patrol the arena. Originally intended to be "walls moving to smush" the player between the walls, but the floor proved easier to implement. When a crusher lands on a player, the player respawns back to their platform. Once all collectibles are collected, the game shows a win screen and both players can vote to restart.

Authority Model

The game uses Fusion 2 Shared Mode exclusively, as required by the module. In Shared Mode, there is no dedicated server, each peer can hold StateAuthority over different NetworkObjects simultaneously. Each player has StateAuthority over their own Player object and controls their own movement. The ScoreManager and GameStateManager are spawned by the first player to join, who holds StateAuthority over them. Collectibles are also spawned by the first player.

All state changes are guarded by HasStateAuthority. For objects where only one peer holds authority (ScoreManager, GameStateManager, Collectibles), other players use RPCs to request state changes. For example, when Player 2 walks into a collectible, the PlayerCollector script sends an RPC to the collectible's StateAuthority (Player 1), which validates and applies the collection.

Synchronisation Approach

[Networked] properties are used for all persistent state: scores (NetworkDictionary<PlayerRef, int>), collectible IsCollected state (NetworkBool), game over state, winner, and restart votes. These are the correct tool because they automatically synchronise to all peers and handle late joiners, a player connecting mid-game receives the current state immediately, although no third party is ever intended to join the game in the present state of development.

RPCs are used for request-style interactions: RpcRequestScore (request score increment), RpcCollect (request collectible pickup), RpcRegisterPlayer (request player registration in scoreboard), and RpcVoteRestart (request restart vote). In each case, the RPC sends from any peer (RpcSources.All) to the StateAuthority (RpcTargets.StateAuthority), which validates the request before modifying [Networked] state.

Issues Encountered

- 1. Hardcoded player identification.** In CA2, I used `player.RawEncoded == 2` to identify players, which was fragile and broke with different join orders. For CA3, I replaced this with a `NetworkDictionary<PlayerRef, int>` that dynamically tracks all players regardless of their raw ID values. The `ScoreManagerCA3` sorts players by `RawEncoded` and assigns display numbers (1, 2, etc.) at render time.
- 2. Double ScoreManager spawning.** In Shared Mode, each client runs `OnPlayerJoined` independently. Both clients initially spawned their own `ScoreManager`, causing overlapping UI. Fixed by adding a delayed check: the client waits 1 second before spawning, then checks if a `ScoreManager` already exists on the network.
- 3. CharacterController blocking position resets.** The `CharacterController` component prevented direct `transform.position` changes for respawning and initial spawn positioning. Resolved by temporarily disabling the `CharacterController`, setting the position, then re-enabling it.
- 4. NetworkTransform overriding spawn positions.** The `NetworkTransform` component was snapping spawned objects back to origin on the first network tick. Fixed by ticking "Disable Shared Mode Interpolation" and re-asserting position in the `Spawned()` callback.
- 5. OnTriggerEnter unreliable with CharacterController.** Trigger-based pickup detection did not fire reliably between `CharacterControllers` and trigger colliders. Replaced with `Physics.OverlapSphere` in a `PlayerCollector` script that runs on each player's authority, actively checking for nearby collectibles each frame.

Robustness and Limitations

The implementation is robust within its two-player scope. Scores synchronise correctly, collectibles disappear on both clients when collected, crusher respawns work, and the game restart cycle functions. The main limitations are: (1) the game is designed for exactly two players, a third player would spawn at the same position as player one; (2) crushers are not networked and run locally on each client, which could cause minor visual differences between clients; (3) there is no server-side validation of collectible pickup order, meaning in theory two players could collect the same item simultaneously in a high-latency scenario.

Authentication Pipeline

The project integrates Unity Authentication with Photon Fusion Custom Authentication and an Azure Function proxy. The authentication flow works as follows:

1. The player presses Join. Unity Services is initialised and Unity Authentication signs the player in anonymously.
2. The client obtains a Unity access token and player ID.
3. The client adds token, playerId, and authKey to Photon auth parameters.
4. Fusion attempts connection. Photon forwards the auth parameters to the Azure Function.
5. The Azure Function validates the shared secret (AUTH_KEY from app settings), checks that token and playerId are present and non-trivial, then returns ResultCode 1 (success) or 2 (failure).
6. Photon either admits the client or rejects with CustomAuthenticationFailed.

A stricter version that validated the Unity token against Unity's backend API was attempted. The Azure Function was updated to call Unity's player-auth endpoint with the token, but the correct REST route could not be verified within the available time. Azure logs confirmed the function reached Unity but received 404 Not Found, establishing that the issue was the endpoint path, not the token or networking. The final submission uses the stable shared-secret version, which provides a complete and demonstrable authentication chain.

The most important architectural decision was moving AUTH_KEY out of source code and into Azure app settings. This means the secret is not embedded in the client build and can be rotated without redeploying code.

Server-side validation via the Azure proxy is necessary rather than client-side validation alone because a client can be modified or spoofed. If the client self-certifies access, any modified client could bypass authentication entirely. By placing the allow/deny decision on a backend that Photon trusts, the authentication cannot be circumvented from the client side.

Section 4: Networking in Modern Engines

Authority and State Synchronisation: Fusion 2 vs Unreal Engine 5

Photon Fusion 2 and Unreal Engine 5 take fundamentally different approaches to networked game architecture. Fusion 2 offers a choice between Host Mode (traditional client-server) and Shared Mode (distributed authority), while Unreal Engine 5 uses a strict server-authoritative model.

In Fusion's Shared Mode, which this project uses, there is no dedicated server process. Each peer can hold `StateAuthority` over different `NetworkObjects` simultaneously. State changes are guarded by `HasStateAuthority` on a per-object basis. In Unreal, the server holds authority over all replicated actors. Clients request changes via Server RPCs, and the server pushes state to clients via replicated properties. Unreal's `HasAuthority()` on the server is the closest equivalent to Fusion's `HasStateAuthority`.

For state synchronisation, Fusion's `[Networked]` properties and Unreal's `UPROPERTY(Replicated)` serve the same purpose: marking values for automatic replication from authority to all peers. Both handle late joiners by sending current state on connection. Unreal adds `ReplicatedUsing` for `OnRep` callbacks when values change on the client, which Fusion approximates through change detection.

RPCs differ in structure. Fusion declares source and target explicitly (`RpcSources.All`, `RpcTargets.StateAuthority`). Unreal categorises RPCs by direction: Server RPCs (client to server), Client RPCs (server to owning client), and `NetMulticast` (server to all). Fusion's approach is more flexible but requires the developer to think about source/target pairs rather than directional categories.

Lag, Latency, and Compensation

Network latency in real-time games comes from several sources: physical distance between peers, network congestion, serialisation/deserialisation overhead, and tick rate mismatches. In a competitive game, even 50-100ms of latency can make the experience feel unresponsive.

Client-side prediction allows the local client to immediately apply input without waiting for server confirmation, making movement feel responsive. When the server state arrives and differs from the prediction, server reconciliation corrects the client's state. Fusion 2 handles this natively in Host Mode through its tick-based simulation, but in Shared Mode (as used in this project), prediction is less relevant because each peer is authoritative over their own objects.

Lag compensation is critical for hit detection in fast-paced games. The server rewinds time to the moment a shot was fired on the client, checking for hits against historical positions. Unreal implements this through its built-in replication and movement prediction systems. Fusion provides interpolation and extrapolation tools for smoothing remote object positions.

In *Crush Arena*, latency is less critical than in a more competitive fighting game because the gameplay is collection-based rather than precision-based. The main latency-sensitive interaction is collectible pickup, where two players might reach the same collectible simultaneously. The authority guard on the collectible's `StateAuthority` resolves this deterministically, whichever request arrives first is processed, and the second is rejected because `IsCollected` is already true.

Implementation Tips from Experience

Several practical lessons emerged from building *Crush Arena*:

Guard every state write. Any `[Networked]` property modification without a `HasStateAuthority` check will cause desyncs in Shared Mode. This was the single most common bug during development.

Use RPCs for requests, `[Networked]` for state. RPCs are fire-and-forget; they do not persist for late joiners. Any value that must survive a reconnect belongs in a `[Networked]` property, with RPCs used only to request changes to that property.

Test with two clients from the start. Bugs that only appear with multiple clients (double spawns, authority conflicts, trigger detection differences) are invisible in single-client testing.

Avoid `runner.IsServer` in Shared Mode. `IsServer` is always false in Shared Mode because there is no server process. Replace with `Object.HasStateAuthority` on the specific object being modified.

Industrial Context and Trends

The games industry increasingly expects junior developers to understand networked game architecture beyond basic client-server communication. GDC presentations from studios working on large-scale multiplayer titles emphasise the importance of understanding authority models, state synchronisation strategies, and the trade-offs between deterministic lockstep and state synchronisation approaches (Aldridge, 2023).

There is a shifting trend moving back and forth on cloud gaming infrastructure (where server-side rendering and streaming reduce client hardware requirements), deterministic lockstep networking for fighting games and RTS titles (where all clients simulate identically from the same inputs), and hybrid approaches that combine aspects of both models. The rise of services like Photon, Netcode for GameObjects, and Unreal's built-in replication reflects an industry trend toward middleware solutions that handle the complexity of networking, allowing developers to focus on gameplay rather than low-level network programming.

For a junior developer, the most important competency is understanding why authority matters, not just how to implement it. A developer who can explain why only the StateAuthority should write to networked state, and what happens when that rule is violated, demonstrates the kind of reasoning that studios value.

Section 5: Version Control and Professional Workflow

My version control practice evolved across the module. In the early weeks, commits were infrequent and clustered around deadlines. The CA1 submission consisted of a single large commit containing all rendering work. By CA2, I had adopted prefixed commit messages (net:, feat:, docs:, fix:) but still committed directly to master.

For CA3, I committed more regularly throughout development, used meaningful messages for each logical change. However, I dialed back on prefixed commit messages. Once I established a base core gameplay loop, I shifted to a new branch to further my networking development, without risking not having a quickly accessible working build to tag as a submit. The most important lesson was verifying which Git repository I was committing to; during CA2, I accidentally committed to the wrong .git directory (the lecturer cloned repo with LFS history) and had to rebuild my commit history. This cost significant time and taught me to always check `git remote -v` before starting work. The experience of rebuilding lost commit history is one I do not intend to repeat.

Going forward into a professional or collaborative project, I would establish a branching strategy from day one, use feature branches for all non-trivial work, and enforce a commit message convention across the team. As while I did not find much use with prefix messages in a solo project, this was mostly due to my own familiarity with each commit. With more frequent commits from others, prefixes and branches gain far more utility than a solo project, where I feel there is limited use case for them.

Section 6: Overall Reflection

The most significant technical challenge I encountered across this module was getting Photon Fusion 2's Shared Mode networking to work correctly with Unity's CharacterController and NetworkTransform components. These systems were not designed with each other in mind, and the interactions between them caused a series of frustrating bugs that consumed more debugging time than any other aspect of the project.

The first problem was spawn positioning. When a player object was spawned via `Runner.Spawn()` at a specific position, the NetworkTransform component would override that position on the next network tick, snapping the player back to origin. The fix was to disable the CharacterController, re-assert the position in the `Spawned()` callback, and re-enable it, but finding this solution required adding debug logging at every stage of the spawn process and systematically eliminating possible causes. The same CharacterController issue appeared again with respawning: calling `transform.position` directly did nothing because the CharacterController intercepts position changes. Again, the solution was to temporarily disable it.

The second major challenge was trigger detection. Unity's `OnTriggerEnter` callback, which I initially used for both collectible pickup and crusher collision, did not fire reliably when one of the colliding objects used a CharacterController. This was particularly confusing because the trigger worked perfectly in single-client testing but failed intermittently with two clients. The eventual solution was to replace trigger-based detection entirely with `Physics.OverlapSphere` (for collectibles) and `Physics.OverlapBox` (for crushers), which check for collisions actively each frame rather than relying on callbacks. This was a valuable lesson in not trusting that Unity's built-in systems will behave identically in all configurations.

The authentication pipeline was the third major challenge, though for different reasons. The technical implementation of Unity Authentication and Photon Custom Authentication was relatively straightforward, the

difficulty was in attempting the stricter server-side token validation against Unity's backend API. I spent several hours debugging Azure Function calls to what I believed was the correct Unity endpoint, only for Azure logs to consistently show 404 Not Found. The debugging process itself was educational: by examining the logs carefully, I could establish that the problem was specifically the endpoint path, not the token format, bearer header, or Azure networking. The decision to revert to the stable shared-secret version for submission was a time-boxing decision, the stricter version was a nice-to-have, not a requirement, and submitting a broken implementation would have been worse than submitting a working simpler one.

The decision I am most satisfied with is the game design itself. The Crush Arena concept, two players competing to collect items while avoiding Thwomp-style crusher NPCs, emerged from thinking about what would create a meaningful interaction between the networking, the NPC AI, and the scoring system. The crushers use NavMesh pathfinding to chase players, which satisfies the NPC requirement while also creating genuine gameplay tension. The elevated spawn platforms with a drop into the arena create a natural safe zone without requiring complex NavMesh carving. And the collectible-based scoring gives both players a clear objective that is simple to understand but competitive to pursue. Every technical system in the project serves the gameplay loop rather than existing as an isolated demonstration.

The Thwomp-style crusher behaviour was a particularly satisfying implementation. The state machine (Patrol -> Chase -> Hover -> Shake -> Crush -> Rise) creates readable, telegraphed attacks that players can learn to avoid. The NavMeshAgent handles ground-plane pathfinding while the visual child object handles the vertical hover/slam animation, keeping the systems cleanly separated. This separation of concerns made debugging much easier because I could test pathfinding and crush behaviour independently.

If I were to begin the project again with the knowledge I now have, I would make three changes. First, I would use Shared Mode from the start rather than beginning with Host Mode in CA2 and having to refactor. The authority model is fundamentally different, and code written for Host Mode (using `runner.IsServer`) does not translate to Shared Mode. Starting in the correct mode would have saved the refactoring time and avoided the CA2 feedback about using the wrong mode.

Second, I would invest time earlier in understanding how `CharacterController` interacts with Fusion's `NetworkTransform`. Almost every major bug in CA3 came from this interaction, spawn positions being overridden, respawns failing silently, triggers not firing. If I had built a small test scene specifically to understand these interactions before starting the main implementation, the development process would have been significantly smoother.

Overall, the module has given me a practical understanding of what it means to build a networked game, not just the API calls, but the reasoning about authority, the debugging of distributed systems, and the professional discipline of version control and documentation. These are skills I expect to carry directly into industry.

Attribution and AI Use Statement

Assets

All primary assets (Jade Toad model, PBR textures) provided through the module GitHub repository. Noise texture (Samples_FractalNoises) from Unity SRP Core samples. Arena geometry built with Unity basics. No additional third-party art assets used.

Packages

Photon Fusion 2 SDK under free educational tier. Unity AI Navigation package. Unity Authentication package. Unity Input System package.

AI Use Statement

Claude (Anthropic) and perplexity.ai was used as a development assistant throughout the project for code guidance and debugging support through console log explanations. All implementation decisions, architecture choices, and reflective analysis are my own. AI-generated code was reviewed, tested, and adapted to fit the project's specific requirements. AI-assisted text was reviewed and edited for accuracy and personal voice.